destroyed, and potentially the storage is even reused for another object. You typically get lifetime issues where a thread references local variables that go out of scope before the thread function has completed, but they aren't limited to that scenario. Whenever the lifetime of the thread and the data it operates on aren't tied together in some way, there's the potential for the data to be destroyed before the thread has finished and for the thread function to have the rug pulled out from under its feet. If you manually call `join()` in order to wait for the thread to complete, you need to ensure that the call to `join()` can't be skipped if an exception is thrown. This is basic exception safety applied to threads.

It's the problematic race conditions that are the killers. With deadlock and livelock, the application appears to hang and become completely unresponsive or takes too long to complete a task. Often, you can attach a debugger to the running process to identify which threads are involved in the deadlock or livelock and which synchronization objects they're fighting over. With data races, broken invariants, and lifetime issues, the visible symptoms of the problem (such as random crashes or incorrect output) can manifest anywhere in the code—the code may overwrite memory used by another part of the system that isn't touched until much later. The fault will then manifest in code completely unrelated to the location of the buggy code, possibly much later in the execution of the program. This is the true curse of shared memory systems—however much you try to limit which data is accessible by which thread and try to ensure that correct synchronization is used, any thread can overwrite the data being used by any other thread in the application.

Now that we've briefly identified the sorts of problems we're looking for, let's look at what you can do to locate any instances in your code so you can fix them.

10.2. Techniques for locating concurrency-related bugs

In the previous section we looked at the types of concurrency-related bugs you might see and how they might manifest in your code. With that